

Formal Verification, Integration and Physical Evaluation of Prime-Field Masking on Silicon

Gaëtan Cassiers^{1,2}, Thorben Moos², Amir Moradi³, Nicolai Müller³
and François-Xavier Standaert²

¹ CryptoExperts, Paris, France

gaetan.cassiers@cryptoexperts.com

² UCLouvain, Louvain-la-Neuve, Belgium

firstname.lastname@uclouvain.be

³ Technische Universität Darmstadt, Darmstadt, Germany

amir.moradi@tu-darmstadt.de, nicolai.mueller1@tu-darmstadt.de

Abstract. The resistance of provably secure masked circuits to physical attacks depends in part on the underlying algebraic group and recombination function. Masking over finite fields of odd prime order has been demonstrated, both in theory and in practice, to provide increased natural resistance to side-channel and fault attacks. Its instantiation with a simple additive encoding and implementation-friendly prime modulus was suggested to lead to favorable tradeoffs between security and performance in prior works. To most efficiently leverage these advantages, a family of lightweight Tweakable Block Ciphers (TBCs) called Feistel for Prime Masking (FPM) has been introduced by Grassi *et al.* at EUROCRYPT’24, together with a first hardware-oriented instance called **small-pSquare**. Yet, barriers for the use and further development of prime-field masking continue to exist and include the lack of automated verification tools compatible with arithmetic over $\mathbb{F}_p$, as well as efficient methods for constant-time generation of uniformly distributed randomness over the field. In this work we tackle these barriers and present our findings from formally verifying, securely integrating and physically evaluating higher-order masked implementations of **small-pSquare** as an exemplary case study. Our integration includes the tape-out of an Application-Specific Integrated Circuit (ASIC) manufactured in 65 nm technology and a custom Printed Circuit Board (PCB). We demonstrate how to securely verify prime-field masked circuits with existing tools such as SILVER, MATCHI and PROLEAD and certify the glitch+transition robustness of our concrete implementations. Along the way we discover and solve a 0-issue originating from incomplete modulo reductions which is present in public source codes of masked prime-field ciphers but has never been discussed. We also introduce Privium, a Bivium-inspired primitive, to efficiently produce random values uniformly distributed over $\mathbb{F}_p$ without the need for rejection sampling. We then describe our efficient serialized pipelined **small-pSquare** architecture enabling an attractive tradeoff between area and latency and compare its pre- and post-layout implementation figures. Finally, we experimentally demonstrate the strong leakage resistance of our formally verified circuits on real silicon.

Keywords: Prime-Field Masking · FPM · Tweakable Block Cipher · small-pSquare · Privium · ASIC · Formal Verification · SILVER · MATCHI · PROLEAD

1 Introduction

Historically, the design of secure cryptographic algorithms and the study of their implementation security have been largely independent research areas. Algorithms would be selected based on their back-box security and implementation efficiency in software and/or

hardware. Their potential protection against physical adversaries was most commonly an afterthought – supposed to be figured out by a different branch of the cryptographic research community. Over the years it has become increasingly apparent that i) physical security can be important in more contexts than traditionally assumed (see e.g. remote side-channel [MDB21] or fault attacks [GMM16]) and ii) considering the suitability of algorithms for the efficient application of the relevant countermeasures as early in the design process as possible can save a lot of headache later and lead to substantially better tradeoffs between security and performance once physical security matters. PICARO [PRC12], Zorro [GGNS13], FIDES [BBK⁺13] and Robin/Fantomas [GLSV14] are early examples of symmetric ciphers deliberately designed for the efficient application of the masking countermeasure. Even recently standardized general-purpose primitives like the ASCON family of lightweight cryptographic algorithms [DEMS21] have made specific design choices (in this case a non-linear layer of low algebraic degree) to stay competitive in settings where the application of masking is a must. Such design goals are more frequently made explicit for symmetric cryptographic algorithms, but examples also exist for asymmetric (post-quantum) cryptography, notably the Raccoon digital signature scheme [dPPRS23, dPKPR24]. The differences between primitive-level and mode-level techniques to provide security in presence of leakage are discussed in [BBC⁺20].

With respect to fault attacks, the tweakable block cipher CRAFT [BLMR19] roots its design decisions in the goal of facilitating the efficient application of countermeasures against Differential Fault Analysis (DFA) – arguably the most powerful class of fault attacks. The authenticated encryption scheme FRIET [SBD⁺20] and the block cipher DEFAULT [BBB⁺21] made use of cipher-level design techniques to inherently detect or prevent fault attacks, but eventually succumbed to vulnerabilities [WFH⁺22, NDE22, JKP24].

In summary, all of the aforementioned primitives make deliberate decisions within their design space to aid the application of side-channel and/or fault countermeasures, commonly observing improved tradeoffs when physical security is needed. While the majority of designs is based on bit-level operations, a number of recent works decided to leave the comfort of this established design space to let physical security considerations fully guide the underlying algebraic setting. This has opened up a new direction for masking-friendly cipher design based on arithmetic over odd prime-order fields which we focus on in this work. First we extend existing formal and simulation-based verification methods to this algebraic setting. Then we integrate and evaluate a concrete primitive in a case study.

Prime Ciphers for Physical Security. After it had been discovered that masking over groups of odd prime order generally comes with improved noise resistance [DFS16], the authors of [MMMS23] performed an information theoretic analysis of different masked encodings to find that equivalent recombination functions provide stronger security for a given number of shares when defined over prime-order fields regardless of the noise level (although the gap to binary-field masking is still most pronounced at lower noise levels). This prompted the design of an exemplary AES-like cipher operating over $\mathbb{F}_p$ with $p = 2^7 - 1 = 127$, a small Mersenne prime. The most straightforward choice for masking-friendly non-linear functions over prime fields is power maps of low degree, where exponentiation can be realized from a few squarings and/or multiplications. Contrary to the common binary-field setting, the squaring operation is non-linear in odd prime fields. Thus, corresponding provably secure and composable gadgets needed to be introduced [CMM⁺23]. These masked gadgets proved to be almost twice as efficient as corresponding masked multiplications in terms of partial products to compute while providing the same algebraic degree, which immediately made them an interesting component for masking-friendly cipher design. This observation provided the necessary foundation for the introduction of the Feistel for Prime Masking (FPM) family of lightweight Tweakable Block Ciphers (TBCs) [GMM⁺24] which uses squaring as the only non-linear component inside a Feistel

structure. A hardware-oriented instance of the family is called **small-pSquare** and also based on the Mersenne prime $p = 2^7 - 1 = 127$. A software-oriented instance has been proposed a year later under the name **mid-pSquare** [BGM⁺25] and operates over $\mathbb{F}_{2^{31}-1}$. The latter is supposed to provide strong side-channel security on 32-bit microcontrollers, specifically, but also leverages the positive scaling trends that come with increasing the underlying field size more generally [MSS24, FMM⁺24, FMM⁺25].

Masking Verification. Masking has been proposed more than 25 years ago as a sound countermeasure against side-channel attacks [CJRR99]. Relevant formalizations since then include the probing model [ISW03], the robust probing model [FGP⁺18] as well as different composability notions based on non-interference [BBD⁺16, CS20]. These formalizations enable designers to not only prove the local security of gadgets for atomic operations (like multiplication or squaring) under physical defaults, but also their secure composition. While these notions make the path towards a securely masked cipher implementation significantly less rocky, maintaining robust probing security through optimizations and composition of multiple parts of a larger algorithm still remains extremely brittle and error-prone. For that reason, a large number of automated tools for masking security verification applicable at different stages of the design process has been introduced over the past years [BBD⁺16, BGI⁺18, BGR18, BBC⁺19, KSM20, GHP⁺21, HB21, CGLS21, BMRT22, MM22, RFSG22, ZMM23, ZCF25, BFG⁺25].

Randomness. Masking schemes consume randomness – at least for the initial sharing of the inputs, but most commonly also during each processing of a non-linear gadget. Exceptions to the latter are usually not generalizable to arbitrary orders in an efficient manner. Depending on the concrete number of shares and the size of the cryptographic algorithm to mask, the number of random elements consumed per clock cycle can be very significant, especially in hardware where parallel computation is common. A multitude of solutions exists for concurrent randomness generation for masked implementations [UHM24], but the vast majority of them focuses on generating uniformly random *bits*, i.e., elements of $\mathbb{F}_2$. This also includes the recent study suggesting that in hardware implementation, unrolled Trivium-like stream ciphers lead to great efficiency for masking randomness generation and sufficient leakage resilience to avoid state recovery [CMM⁺24].

1.1 Our Contributions

In this work we provide a comprehensive end-to-end security verification and full integration of masked **small-pSquare** implementations for multiple protection orders on actual silicon, including a physical evaluation of the final circuits on the manufactured chip. To the best of our knowledge this work is the first example of such a complete end-to-end study. As a general contribution we detail how existing formal and simulation-based verification tools, in particular SILVER¹ [KSM20], MATCHI² [Cas25] and PROLEAD³ [MM22], can be used to correctly verify circuits operating over $\mathbb{F}_p$ elements for their robust probing security and trivial composability. During the verification we notice that existing source codes for $\mathbb{F}_p$ squaring gadgets use incomplete modulo reductions in a very liberal, potentially dangerous, manner. We discuss and provide efficient workarounds for this issue and verify their correctness with multiple tools. We then introduce Privium, a Bivium-inspired constant-time generator of uniformly distributed field elements without the need for rejection sampling. The concrete instance proposed is specific to $\mathbb{F}_{2^7-1}$, but its underlying design principles are generalizable to other Mersenne primes. While we do not provide

¹<https://github.com/Chair-for-Security-Engineering/SILVER>

²<https://github.com/cassiersg/matchi>

³<https://github.com/ChairImpSec/PROLEAD>

any black-box security analysis of the Privium primitive in this work, we have tested the quality of its randomness output using common statistical test suites and conjecture its cryptanalytic security and leakage resilience to be stronger than that of Bivium. We leave the further analysis of this conjecture as an interesting scope for further research. Once we have verified that our efficient combination of Privium and masked F-serialized two-stage pipelined implementations of **small-pSquare** satisfies glitch+transition-robust probing security we implement the cores for $d = 2, 3$ and 4 shares on a 65 nm ASIC prototype and perform a leakage detection experiment on practical power measurements. We conclude that verifying and using masked prime-field ciphers is practical today and can lead to favorable tradeoffs between cost and security.

In summary:

- *General Contributions:* In Section 2.2.1, we provide a generic recipe for the formal verification of the trivial composability notion (glitch-robust) t-Probe-Isolating Non-Interference (t-PINI) of any masked gadget operating over any finite field. Additionally, we demonstrate how to verify the glitch+transition-robust probing security of full masked circuits for any field. This verification can be exhaustive when the circuit is composed entirely from t-PINI gadgets (Section 2.2.2) or based on simulations when it is not (Section 2.3). Beyond prime-field masking, our results are expected to be useful for the verification of masked gadgets in post-quantum schemes, too. Yet, verification generally scales poorly to the parameter sizes typically employed in asymmetric cryptography.
- *Partly Generalizable Contributions:* We demonstrate how to efficiently generate uniformly distributed field elements over an odd prime field in hardware. Our concrete example is focused on $\mathbb{F}_{2^7-1}$, but the design principle is generalizable to other Mersenne primes. We also discuss an interplay between the distribution produced by the randomness generator and efficient implementations of field arithmetic that is relevant for any masked circuit defined over a Mersenne prime field.
- *Specific Contributions:* We implement securely masked circuits of the **small-pSquare** TBC on a 65 nm ASIC and evaluate their leakage resistance experimentally. We also discover and repair a flaw in existing masked implementations.

2 Tool-Assisted Verification

We begin our investigation by discussing how masked implementations operating over prime fields can be verified using existing automated tools. We separate between formal verification of security properties at gadget and composition level as well as simulation-based verification of circuits.

2.1 Preliminaries

In this work, we consider the security of masked implementations in the robust probing model [FGP⁺18], where an adversary can place t ϵ -extended probes in the masked circuit. These ϵ -extended probes are convenient abstractions to capture physical defaults such as glitches, transitions and couplings. In this work we focus on glitch-extended probes only, as the resulting security properties at the gadget level are sufficient for our purposes to later (under some additional conditions) obtain glitch+transition-robustness at the composition level. Each glitch-extended probe placed on a wire in the circuit produces as leakage the collective values carried by all wires in the combinatorial circuit that computes the probed wire. Given a set of glitch-extended probes P in a masked gadget over a field $\mathbb{F}_q$, the input

sharings $\mathbf{x}$ and the input randomness $\mathbf{r}$, we denote by $\mathcal{V}_P(\mathbf{x}, \mathbf{r})$ the values observed by the probes, as a map from P to $\mathbb{F}_q$. For such probes, we can define the notion of simulatability, which intuitively means that the probes only depend on some input shares, and not on others.

Definition 1 (Simulatability [BBD⁺16, BBP⁺16]). Let G be a masked gadget over $\mathbb{F}_q$ with input randomness $\mathbf{r} \in \mathbb{F}_q^{n_r}$ and input shares $\mathbf{x} \in \mathbb{F}_q^{n_i}$. A set of glitch-extended probes P in G can be simulated by a set of input shares $I \subset \mathbb{N}$ if there exist a randomized function $\mathcal{S}$ named the simulator such that for any input $\mathbf{x} \in \mathbb{F}_q^{n_i}$,

$$\mathcal{V}_P(\mathbf{x}, \mathbf{R}) \approx \mathcal{S}((\mathbf{x}_i)_{i \in I})$$

where $\mathbf{R}$ is a random variable uniformly distributed over $\mathbb{F}_q^{n_r}$ and $\approx$ denotes the equality of distributions.

This simulation notion was originally introduced for the Non-Interference (NI) and Strong Non-Interference (SNI) security definitions [BBD⁺16]. It was then adopted by the Probe-Isolating Non-Interference (PINI) security definition [CS20] which enables trivial compositional reasoning for masked functions, even in presence of physical defaults such as glitches [CGLS21]. Any gadget that can be written as a composition of PINI gadgets is itself PINI. The trivial share-wise implementation of linear functions is PINI.

Definition 2 (Glitch-robust probe-isolating non-interference [CGLS21]). Let P be a set of t_1 glitch-extended probes in a gadget G and B be a set of share indices (the index of a share is its position in a sharing, starting at 0). The gadget G is glitch-robust t probe-isolating non-interferent if, for any P and B such that $|P| + |B| \leq t$, there is a set A of at most $|P|$ share indices such that the probes in P and glitch-extended probes on the output shares of G with index in B can be simulated by all the input shares of G with index in $A \cup B$.

Some recent works additionally integrate further physical defaults such as registers' transitions into their analysis [CS21, KMMS22, KSM22, MKSM22, MCS22]. As mentioned above, we do not require this for our work, as under some conditions, pipelined glitch-robust PINI gadgets allow secure composition under glitch+transition leakage [CS21].

2.2 Formal Verification

Formal verification tools for gadget- or circuit-level analysis of local and/or compositional security properties include REBECCA [BGI⁺18], **maskVerif** [BBC⁺19], SILVER [KSM20], COCO [GHP⁺21], COCOALMA [HB21], IronMask [BMRT22], VERICA [RFSG22], Prover [ZCF25] and INDIANA [BFG⁺25]. Automated tools for the formal verification of full circuits composed from gadgets satisfying certain composability notions include **maskComp** [BBD⁺16], **tightPROVE** [BGR18], **fullVerif** [CGLS21] and MATCHI [Cas25]. In this work we focus on SILVER for gadget-level verification of glitch-robust trivial composability (PINI) properties and MATCHI for the composition-level verification of the glitch+transition-robustness of the final pipelined circuits.

2.2.1 Gadget-Level Formal Verification

In this section, we want to verify that our masked gadgets are indeed glitch-robust PINI. For this purpose, we use the verification tool SILVER [KSM20]. Since the tool is taking as input a netlist in Hardware Description Language (HDL), SILVER allows us to directly verify the implementation of the gadgets, avoiding any possible mismatch between a verified specification and the concrete implementation. However, natively, SILVER can only verify glitch-robust PINI for circuits masked over $\mathbb{F}_2$.

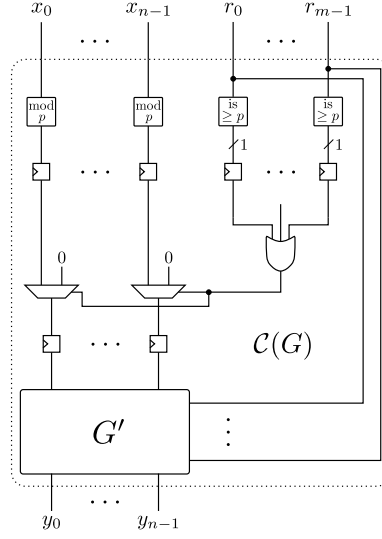


Figure 1: Simple wrapper that can be used to verify synthesized circuits G' of prime-field masked gadgets G using formal verification tools like SILVER. G receives data inputs $x_0, \dots, x_{n-1} \in \mathbb{F}_p$ and random inputs $r_0, \dots, r_{m-1} \in \mathbb{F}_p$ and produces data outputs $y_0, \dots, y_{n-1} \in \mathbb{F}_p$.

We solve this issue by building a circuit compiler $\mathcal{C}$ such that for any gadget G over $\mathbb{F}_p$, $\mathcal{C}(G)$ is a masked circuit over $\mathbb{F}_2$, and $\mathcal{C}(G)$ is glitch-robust t -PINI *if and only if* G is glitch-robust t -PINI. The gadget G must contain only registers and combinational gates for addition, subtraction and multiplication (including squaring as a special case of multiplication).

The first step of the compiler is a simple synthesis from $\mathbb{F}_p$ to $\mathbb{F}_2$: each wire (resp. register) in G is replaced by $\lceil \log_2(p) \rceil$ wires (resp. registers), and each arithmetic gate is replaced by a combinational circuit implementing the arithmetic operation. The synthesized circuits for arithmetic gates are said to be *complete* if any of their output wires is connected to all the input wires. This synthesized gadget is named G' . After this synthesis step, we wrap G' as shown in Figure 1 to get $\mathcal{C}(G)$: for each set of input wires that correspond to an input random r_i of G , we perform the comparison $r_i \geq p$, put a register on the result, and compute the OR of these bits for all randomness inputs. Let us name the resulting wire s . In parallel, each set of input wires that corresponds to an input share x_i of G , is reduced mod p and passes through a register (needed only for synchronization). Then we gate all the input shares: if $s = 1$, all input share bits are replaced by 0 with a MUX⁴, whose output is passed through a register, and we name the results x'_i (each x'_i is a bundle of $\lceil \log_2(p) \rceil$ wires). Finally, we connect G' to the input shares x'_i and to the randomness r_i . The output shares are the outputs of G' . Overall, we denote by $\mathcal{C}(x_i)$ (respectively $\mathcal{C}(y_i)$) the input (resp. output) wires of $\mathcal{C}(G)$ that correspond to the input x_i (resp. output y_i) of G .

Let us now prove that $\mathcal{C}(G)$ is PINI *if and only if* G is PINI, such that we can verify $\mathcal{C}(G)$ through SILVER instead of G .⁵

Theorem 1. *Let G be a masked pipelined gadget over $\mathbb{F}_p$. If the compiler $\mathcal{C}$ uses complete circuits for the synthesis of all arithmetic gates, then G is t -glitch-robust PINI if and only if $\mathcal{C}(G)$ is t -glitch-robust-PINI.*

⁴We could use an AND gate here instead of the MUX, if only replacement by 0 is needed. However, as we demonstrate later, in some cases replacement by other values can be valuable.

⁵This technique also works for verifying other simulatability-based notions such as NI or SNI, although we only prove PINI for the sake of simplicity.

Proof. Let P be a set of probes in G and B be a set of share indices such that $|P| + |B| \leq t$. We assume without loss of generality that any probe in P is the output of an arithmetic gate (outputs of registers can be moved back to their inputs, and input probes do not cause failure of PINI simulation). We find an equivalent set of probes P' in $\mathcal{C}(G)$ by taking for each $p \in P$, an output wire of the synthesized circuit that corresponds to the gate in G that computes p . Assuming that $\mathcal{C}(G)$ is t -glitch-robust PINI, there exists a set of share indices A with $|A| \leq |P|$ such that P' and the output shares of $\mathcal{C}(G)$ with share index in G (let us denote those wires $\mathcal{C}(y_O)$) can be simulated using the inputs of $\mathcal{C}(G)$ with share index in $A \cup B$ (let us denote those wires $\mathcal{C}(x_I)$). That is, there exists $\mathcal{S}'$ such that for any $\mathcal{C}(\mathbf{x})$ and any z ,

$$\Pr [\mathcal{V}_{P' \cup \mathcal{C}(y_O)}(\mathcal{C}(\mathbf{x}), \mathbf{R}') = z] = \Pr [\mathcal{S}'(\mathcal{C}(x_I)) = z],$$

where $\mathbf{R}'$ follows a uniform distribution over $\mathbb{F}_2^{n_r \lceil \log_2(p) \rceil}$. We can rewrite this equation as

$$\begin{aligned} & \Pr [\mathcal{V}_{P' \cup \mathcal{C}(y_O)}(\mathcal{C}(\mathbf{x}), \mathbf{R}') = z \mid \mathcal{V}_s(\mathcal{C}(\mathbf{x}), \mathbf{R}') = 1] \Pr [\mathcal{V}_s(\mathcal{C}(\mathbf{x}), \mathbf{R}') = 1] \\ & + \Pr [\mathcal{V}_{P' \cup \mathcal{C}(y_O)}(\mathcal{C}(\mathbf{x}), \mathbf{R}') = z \wedge \mathcal{V}_s(\mathcal{C}(\mathbf{x}), \mathbf{R}') = 0] = \Pr [\mathcal{S}'(\mathcal{C}(x_I)) = z]. \end{aligned} \quad (1)$$

Further, let use pose

$$\alpha \triangleq \Pr [\mathcal{V}_s(\mathcal{C}(\mathbf{x}), \mathbf{R}') = 1] = 1 - \left(\frac{p}{\lceil \log_2(p) \rceil} \right)^{n_r},$$

and remark that $\Pr [\mathcal{V}_{P' \cup \mathcal{C}(y_O)}(\mathcal{C}(\mathbf{x}), \mathbf{R}') = z \wedge \mathcal{V}_s(\mathcal{C}(\mathbf{x}), \mathbf{R}') = 0]$ is independent of $\mathbf{x}$, hence we denote it by $\beta(z)$. As a last remark, the distribution of $\mathcal{V}_{P' \cup \mathcal{C}(y_O)}(\mathcal{C}(\mathbf{x}), \mathbf{R}') \mid \mathcal{V}_s(\mathcal{C}(\mathbf{x}), \mathbf{R}') = 1$ is identical to $\mathcal{V}_{P \cup y_O}(\mathbf{x} \bmod p, \mathbf{R})$ where $\mathbf{R}$ is uniformly distributed over $\mathbb{F}_p^{n_r}$. Therefore, we can rewrite Equation 1 as

$$\Pr [\mathcal{V}_{P \cup y_O}(\mathbf{x} \bmod p, \mathbf{R}) = z] \alpha + \beta(z) = \Pr [\mathcal{S}'(\mathcal{C}(x_I)) = z].$$

Thus, for any $\mathbf{x}$ such that all x_i satisfy $x_i < p$, we have

$$\Pr [\mathcal{V}_{P \cup y_O}(\mathbf{x}, \mathbf{R}) = z] = (1/\alpha) (\Pr [\mathcal{S}'(\mathcal{C}(x_I)) = z] - \beta(z)),$$

which shows that P and O_y can be simulated while knowing only the input shares x_I , and therefore the G is glitch-robust t -PINI.

The converse implication is proven similarly for the probes within G' : the inputs x'_i being reduced mod p , the simulator for G can be used, while if the randomness is larger than p , the probes are independent of $\mathbf{x}'$ since the inputs of G' are 0. It remains to show that the probes in the wrapper circuit do not break the PINI property. For the modular reductions of the x_i and the comparisons of the randomness to p , these probes leak at most one input share (resp. one input random), hence they can be simulated by replacing them with probes on the corresponding inputs of G' . Finally, we consider glitch-extended probes on the outputs of the MUXes. Each such probe observes one x_i and the results of all comparisons. The value of x_i can be obtained by requesting the corresponding input share index as input of the simulator. Then, the results of the comparison can be revealed since they do not reveal anything about the shares: if $s = 1$, then there is no secret leakage inside G , while all comparisons output 0 if $s = 0$. $\square$

Using the theoretical circuit compiler in practice is as simple as manually (or automatically, which we leave as an interesting task for future work) instantiating the wrapper depicted in Figure 1 around G' . Results of our gadget verification are presented in Section 3.

2.2.2 Composition-Level Formal Verification

In addition to the local and compositional properties of our gadgets, we ideally would like to perform an automated verification of the final composite circuits. MATCHI [Cas25] is a recent tool for this purpose, taking as input a netlist and its respective input signals, to simulate the execution of the synchronous circuit, tracking the dataflow and dependencies of masked sharings and random inputs in each wire. Non-linear gadgets have to be annotated explicitly by their PINI/OPINI property, which MATCHI assumes to be correct. Based on this process the tool then evaluates the glitch+transition-robust probing security of the circuit. Interestingly, MATCHI is not inherently limited to $\mathbb{F}_2$ verification and can handle synthesized circuits of implementations defined over $\mathbb{F}_p$. Yet, the correct distribution of the inputs (data and randomness) according to the underlying prime field is not explicitly verified and has to be guaranteed by the user. The user must only generate a netlist of their masked implementation, correctly annotate the included PINI gadgets and their inputs and outputs within the netlist, provide a testbench following the correct input distributions and the verification can be performed. The full small-pSquare implementations that successfully pass this check are described in Section 4.

2.3 Simulation-Based Circuit Verification

While formal verification tools can prove the conformity of circuits to specific security notions, e.g., glitch-robust PINI, by exhaustively analyzing all possible circuit inputs, outputs and internal states, the computational complexity of these checks usually scales poorly to larger implementations, like fully masked cipher cores, or higher protection orders. Therefore, incomplete evaluation approaches based on statistical analysis of a restricted set of circuit states have emerged. These simulation-based tools can provide, at least to some extent, evidence of the overall security of larger masked designs. Tools of this family include PROLEAD [MM22] and PROLEAD_SW [ZMM23]. Specifically, the PROLEAD tool assesses the security of a hardware circuit, represented as a gate-level netlist, under the robust probing model by leveraging gate-level simulations and statistical independence testing [MM22]. Since PROLEAD does not simulate all possible circuit states, its reliability primarily depends on the number of simulations performed, that is, the error probability decreases as more simulations are conducted. While PROLEAD was originally designed to evaluate Boolean masked circuits, i.e., by sampling Boolean masked test vectors for simulation, as successfully demonstrated in various works, its application to other masking schemes is relatively new. Since the release of version three, PROLEAD has supported the evaluation of masking in finite fields beyond $\mathbb{F}_2$. Specifically, PROLEAD has been extended to sample test vectors masked in other finite fields, including odd prime fields, making it well-suited for evaluating the designs considered in this work. To use this feature, the user only needs to specify the finite field of the input and output shares. PROLEAD then automatically interprets its simulation groups as vectors of elements in the chosen finite field. For example, when using prime-field masking over $\mathbb{F}_{2^2-1}$, a simulation group such as 4'b\$\$\$\$ no longer represents four independent random bits, but instead two concatenated elements of $\mathbb{F}_{2^2-1}$ which can subsequently be assigned bit-wise to the individual signals of the implementation under evaluation. Yet, regardless of the finite field under consideration, PROLEAD models formal adversaries with the ability to probe a single wire, i.e., an element in $\mathbb{F}_2$, rather than a single element from the underlying finite field, as specified by the robust probing model. Consequently, when the masking is not performed in $\mathbb{F}_2$, PROLEAD evaluates a slightly weaker definition of probing security, known as bit-level security.

3 Gadget Verification, Incomplete Reductions, 0-Issue

With a concrete verification strategy now in place, we can apply the developed methodology to analyze the security of existing gadget instances. Hardware implementations of masked squaring gadgets over Mersenne prime fields as well as full cipher implementations have been previously published in conjunction with [CMM⁺23]⁶ and [GMM⁺24]⁷. Interestingly, in both previous works, the implementations make liberal use of incomplete modulo reductions without clear documentation or guidelines. We now argue that this is dangerous and, in some cases, leads to circuits that suffer from security order reductions unless certain non-intuitive conditions are observed by the provided randomness inputs. We then discuss how to remedy this situation at essentially no cost.

The common Mersenne prime reduction trick for $p = 2^n - 1$ consists in adding the lower n bits of a variable x which is to be reduced, to its upper 1-to- n bits (depending on how large x is), resulting in $x' = x \bmod p$. If the initial value x turns out to be a multiple of p , a complete modulo reduction should produce the output 0. However, the addition-based reduction trick only results in a 0 output if the original value x was already 0 before reduction, i.e., $x' = x = 0$. In all other cases where x' should evaluate to 0, the result will be p which fits into n bits and is thus not further reduced (even when repeating the procedure). To implement a complete modulo reduction, it would be necessary to check if the obtained n bits after reduction are equal to p and then replace it by 0. Of course, this additional step incurs an overhead in form of extra instructions in software and in form of additional area, delay and power consumption in hardware. We assume that this final check has been dropped from prior implementations for such efficiency reasons. Functionally, this is completely unproblematic as $x' = p$ is small enough to fit the required data width and further computations can normally continue without risking an overflow. However, for masked implementations, we need to be more careful.

Concretely, in the EUROCRYPT'24 artifact of [GMM⁺24], the hardware modules used for addition and subtraction modulo a Mersenne prime are implemented as depicted in the column titled "version α " of Table 1. As explained above, the output c of the addition module can only become 0 when both inputs a and b are 0. Obviously, if used for randomness addition in a masked gadget, this can lead to trivial insecurity if inputs are supplied in the standard representation of $\mathbb{F}_p$ field elements, namely $a, b \in \{0, \dots, p-1\}$. Even if output c passes through a register stage, as is common after randomness additions, a single probe on the output then reveals whether the value to be blinded was already 0 before the randomness addition. In such cases, the probing security of the gadget would be violated. Yet, if in all inputs, the 0-element is actually represented by p ($11 \dots 1_2$ in binary representation) instead of 0 ($00 \dots 0_2$ in binary representation), leading to $a, b \in \{1, \dots, p\}$, this problem disappears and distributions remain uniform. We assume this to be the intended usage of these concrete squaring gadget implementations, as the physical security analysis of prior works should have revealed this leakage otherwise. However, this simple fix becomes insufficient in circuits where the modular addition is paired with a modular subtraction behaving similarly, see Table 1.

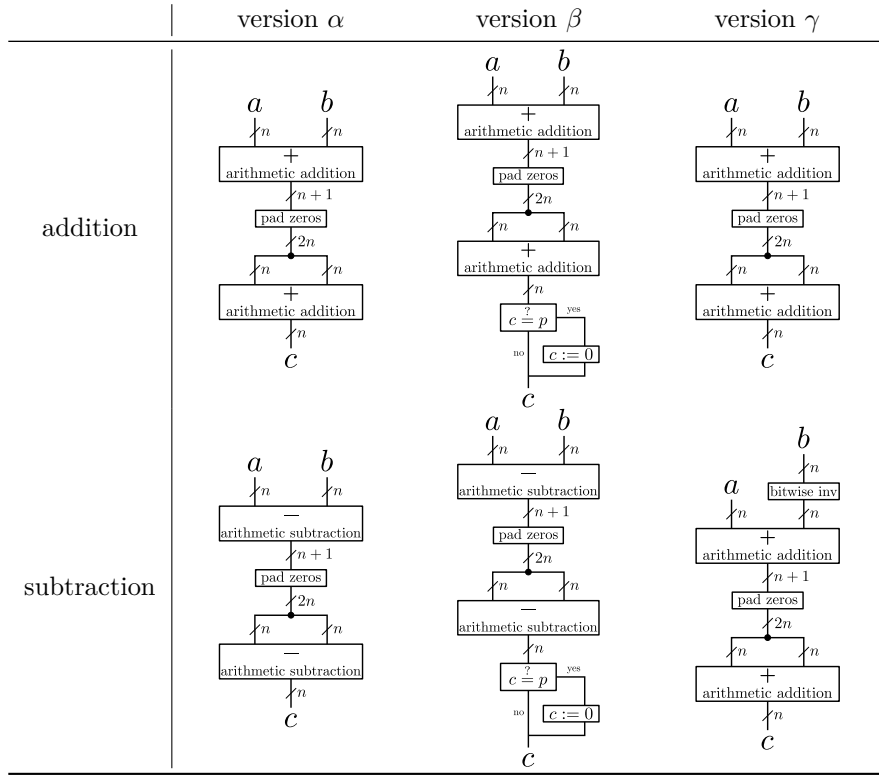
Subtraction module version α outputs $c = 0$ whenever $a = b$. However, it only outputs $c = p$ when $a = p$ and $b = 0$. While the latter case cannot natively occur when using input distributions $a, b \in \{1, \dots, p\}$, it may occur when composing multiple addition and subtraction modules to create a larger circuit. For example, if both inputs to the addition module described above are outputs of the subtraction module described above, the final addition result can become 0 and therefore fall outside the intended distribution, leaking about multiple intermediate values, even through register stages.

We have devised two straightforward methods to prevent this issue from occurring

⁶https://github.com/uclcrypto/prime_field_masking_hardware

⁷<https://github.com/uclcrypto/small-pSquare>

Table 1: Implementations of addition and subtraction modulo a Mersenne prime.



and analyze their efficiency in the following. We first denote the variant of the public squaring gadgets using the previously described modular addition and subtraction functions as version α . Version β simply performs complete modulo reductions after arithmetic operations, see Table 1. With all modular arithmetic using complete reductions, no security issues are expected to arise, but this change increases the cost of the circuit. Hence, as a cheaper alternative we can keep the addition module version α as long as we do not pair it with the corresponding subtraction module version α , or in fact any module that can result in a 0 output despite its inputs being distributed uniformly over $\{1, \dots, p\}$. To devise an alternative subtraction module, we can leverage the observation that inverting all bits of an element from a Mersenne prime field in binary representation, yields the additive inverse of that field element. It is easy to see that two values with inverted bits at each location result in the 1-vector ($11 \dots 1_2$) when summed together, which represents p and therefore is supposed to be reduced to 0. Hence, if instead of using a module based on actual arithmetic subtraction, we simply obtain the additive inverse of the second operand and use addition instead, the result can never become 0 if the input distributions $\{1, \dots, p\}$ are observed. The corresponding subtraction arithmetic, internally based on inversion and addition, is denoted as version γ in Table 1.

In Table 2 we summarize the cost and performance differences between the three gadget versions α , β , γ for $d = 2, 3$ and 4 shares, respectively. Table 3 shows the results of our analysis of the conformity of the gadgets to the security notions glitch-robust $(d - 1)$ th-order probing security and PINI composability. We distinguish between different input distributions $x_i, r_j \in \{0, \dots, p - 1\}$ and $x_i, r_j \in \{1, \dots, p\}$ to highlight their impact on security. It is important to note that realizing the latter input distribution required corresponding changes to the wrapper that is used for SILVER verification shown in Figure 1. In particular, the modulo reduction at the input needs to map 0 to p and the checks of the randomness need to trigger whenever the random input is 0. Similarly,

Table 2: Cost and performance comparison of the different squaring gadget variants over $\mathbb{F}_p$ for $p = 2^7 - 1$.

d	version	Area [GE]		Delay [ns]		Power [mW]	
		abs.	rel.	abs.	rel.	abs.	rel.
2	α	1266.00	100.0 %	4.21	100.0 %	0.0636	100.0 %
	β	1321.50	104.4 %	4.49	106.7 %	0.0653	102.7 %
	γ	1267.25	100.1 %	4.21	100.0 %	0.0636	100.0 %
3	α	3259.25	100.0 %	3.49	100.0 %	0.1785	100.0 %
	β	3404.00	104.4 %	3.85	110.3 %	0.1830	102.5 %
	γ	3265.25	100.2 %	3.49	100.0 %	0.1788	100.2 %
4	α	5431.50	100.0 %	5.62	100.0 %	0.2932	100.0 %
	β	5732.25	105.5 %	6.98	124.2 %	0.3026	103.2 %
	γ	5435.25	100.1 %	5.62	100.0 %	0.2937	100.2 %

Table 3: Verification results of glitch-robust $(d - 1)$ th-order probing and PINI security of the different squaring gadget variants over $\mathbb{F}_p$ for $p = 2^n - 1$.

d	version	PINI ¹	$x_i, r_j \in \{0, \dots, p-1\}$			PINI ¹	$x_i, r_j \in \{1, \dots, p\}$		
			exponents	probing ²	exponents		exponents	probing ²	exponents
2	α	$\times$	$2 \leq n \leq 5$	$\times$	$2 \leq n \leq 7$	$\checkmark$	$2 \leq n \leq 5$	$\checkmark$	$2 \leq n \leq 7$
	β	$\checkmark$	$2 \leq n \leq 5$	$\checkmark$	$2 \leq n \leq 7$	$\checkmark$	$2 \leq n \leq 5$	$\checkmark$	$2 \leq n \leq 7$
	γ	$\times$	$2 \leq n \leq 5$	$\times$	$2 \leq n \leq 7$	$\checkmark$	$2 \leq n \leq 5$	$\checkmark$	$2 \leq n \leq 7$
3	α	$\times$	$2 \leq n \leq 5$	$\times$	$2 \leq n \leq 7$	$\times$	$2 \leq n \leq 5$	$\times$	$2 \leq n \leq 7$
	β	$\checkmark$	$2 \leq n \leq 5$	$\checkmark$	$2 \leq n \leq 7$	$\checkmark$	$2 \leq n \leq 5$	$\checkmark$	$2 \leq n \leq 7$
	γ	$\times$	$2 \leq n \leq 5$	$\times$	$2 \leq n \leq 7$	$\checkmark$	$2 \leq n \leq 5$	$\checkmark$	$2 \leq n \leq 7$
4	α	$\times$	$2 \leq n \leq 5$	$\times$	$2 \leq n \leq 7$	⁻³	-	$\checkmark$	$2 \leq n \leq 7$
	β	⁻³	-	$\checkmark$	$2 \leq n \leq 7$	⁻³	-	$\checkmark$	$2 \leq n \leq 7$
	γ	$\times$	$2 \leq n \leq 5$	$\times$	$2 \leq n \leq 7$	⁻³	-	$\checkmark$	$2 \leq n \leq 7$

¹ Formal verification using SILVER.² Simulation-based verification using PROLEAD (no exhaustive check).³ Check not completed due to excessive runtime.

PROLEAD does not natively support randomness distributions of this kind, which made a simple wrapper module necessary to consistently add 1 to all input values.

The glitch-robust PINI security is verified using SILVER for all 2- and 3-share gadgets. The table also shows the different Mersenne exponents that could be verified. Larger exponents, and for some of the 4-share gadgets even the smallest Mersenne primes $p = 2^2 - 1$ could not be evaluated, either because of tool or computational limitations (e.g., more than 100 hours on a 256 GB RAM and 128 vCPUs machine). Thus, we had to resort to verifying the glitch-robust probing security of the gadgets with PROLEAD. Here also, the tested Mersenne exponents are specified. Please note that while PROLEAD can handle much larger circuits than SILVER, the statistical biases in the distributions may become so small for larger Mersenne prime exponents (e.g., ≥ 13) that prohibitively many simulations would be needed to discover them. Version α guarantees PINI and probing security under glitches for 2 shares, if the correct input distribution is chosen. At 4 shares, no probing security violation was discovered for the correct input distribution, but the 3 share gadget is insecure regardless as it contains combinations of additions and subtractions that cause security issues. Version β succeeds all checks that could be executed, regardless of how the 0-element is represented (either 0 or p). This is expected as the β version always performs complete modulo reductions, which comes at a small area ($\leq 5.5\%$) and power ($\leq 3.2\%$) overhead, in addition to a more notable delay increase ($\leq 24.2\%$). Version γ expectedly succeeds all checks where the 0-element is represented by p . While the γ version performs incomplete modulo reductions only, it makes sure that if no input contains the 0-vector, no intermediate value can result in the 0-vector either, making distributions uniform and consistent, thereby preserving security. The overhead of the γ version compared to the

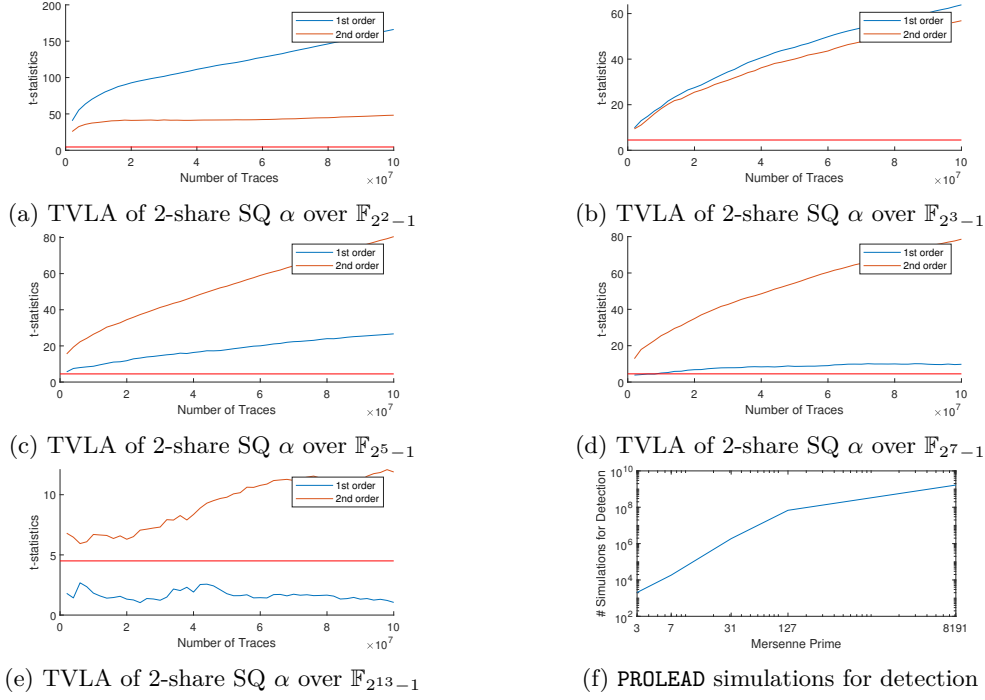


Figure 2: Test Vector Leakage Assessment (TVLA) results of first-order masked squaring gadgets of version α and a corresponding PROLEAD estimation of the number of simulations needed to detect first-order leakage.

insecure α version is negligible ($\leq 0.2\%$ in area and power, 0% in delay). Hence, we consider this version for our efficient masked hardware implementations of **small-pSquare**, described in the next section.

3.1 Empirical Verification

To empirically assess the severity of the 0-issue in practice, we have performed FPGA measurements and leakage detection tests. The bias introduced by not handling the case of p or 0 correctly obviously depends on the size of the Mersenne prime chosen. For small Mersenne primes, the bias is strong as input words frequently carry the problematic value when sampling randomly, while for larger Mersenne primes the bias decreases. To investigate this relationship, we have measured 100 million power consumption traces of different first-order masked squaring gadgets of version α on a Sakura-G [sak] side-channel evaluation board. The probe and oscilloscope used are described in Section 5. First- and second-order Test Vector Leakage Assessment (TVLA) results of gadgets defined over $\mathbb{F}_{2^n-1}$ for exponents $2 \leq n \leq 13$ are depicted in Figure 2. It is clearly noticeable that the magnitude of the first-order t -statistic decreases and the number of traces to overcome the 4.5 threshold increases with growing prime size. For reference, we also performed the same experiments in form of noise-free simulations within PROLEAD, confirming the same trend. While from $n = 13$ onwards the biases only become detectable after billions of traces, for $n \leq 7$ the detectable leakage arising from incomplete modulo reductions is significant and should not be ignored in practice.

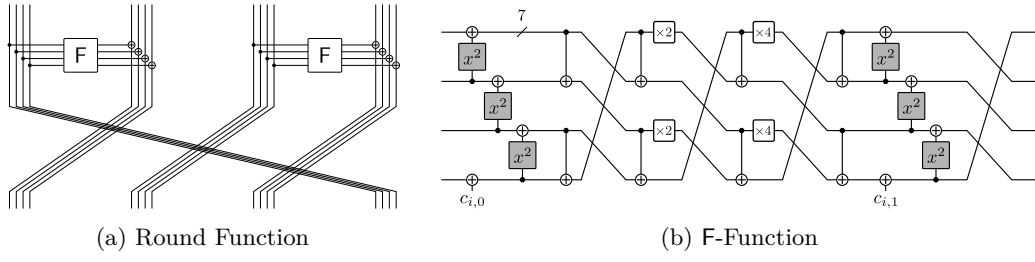
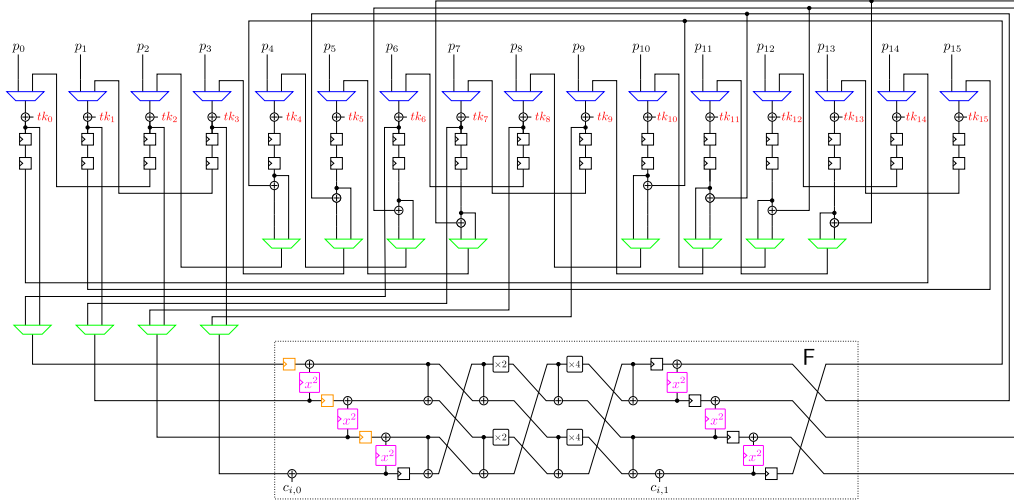


Figure 3: The small-pSquare lightweight Tweakable Block Cipher (TBC) [GMM+24].

Figure 4: F-serialized two-stage pipelined implementation of small-pSquare performing two masked encryptions in $256 + 2 = 258$ clock cycles.

4 Implementation and Integration

In this section we describe the implementation and integration of our masked small-pSquare circuits, based on the formally verified composable gadget instances we just analyzed.

4.1 Serialized Pipelined Implementation

Figure 3 shows the structure of the lightweight Tweakable Block Cipher (TBC) small-pSquare, which is an instance of the Feistel for Prime Masking (FPM) family, introduced at EUROCRYPT’24 [GMM+24]. At the round level and F-function level it is based on Type-II and Type-III generalized Feistel networks, respectively. Its only non-linear components are the 6 squaring operations inside the F-function, two layers of three each. In this work we only consider the variant with $\tau = 1$, i.e., 112-bit state size, 112-bit key size, 112-bit tweak size and 64 cipher rounds for either encryption or decryption with a 112-bit security claim. Using the single-cycle masked squaring gadgets with 2 and 3 shares from [CMM+23] and 4 shares from [GMM+24] we are able to implement a fully masked F-function in only two cycles. We decided to implement only one F-function instance in order to save area and reduce the randomness requirements per cycle and obtained the F-serialized two-stage pipelined implementation of small-pSquare depicted in Figure 4.

The implementation is based on a carefully crafted rotation by 4 state words (which are elements of $\mathbb{F}_{2^{7-1}}$) at a time through the two-stage pipelined F-function to concurrently perform the permutation layer of the top-level Type-II Feistel. In this architecture, two

distinct (plaintext, key, tweak)-tuples can be supplied consecutively in the first 2 cycles. The circuit then performs both encryptions over the next 256 cycles, leading to a total of 258 cycles for 2 **small-pSquare** executions. Important details about the implementation and its depiction are given below.

- The **blue**-colored multiplexers in Figure 4 are all controlled by the same digital signal used to select the round input between global input (first two cycles) and round output (after the first two cycles).
- The **lime**-colored multiplexers in Figure 4 are all controlled by the same digital signal which toggles every two cycles, starting from zero.
- The **red**-colored tk_i inputs in Figure 4 hold tweak elements during the first two cycles of each round and hold zero otherwise. The (unmasked) tweak schedule computation is not shown in this figure for simplicity.
- The **orange**-colored registers are redundant and actually not present in the implementations. They are only shown for better readability of the figure and replaced by additional wiring/multiplexing (slightly cheaper) in practice.
- The **purple**-colored squaring gadgets need to include one internal register stage for all inputs to realize the two-stage pipeline. We use the glitch-robust masked single-cycle PINI squaring gadgets with 2 and 3 shares from [CMM⁺23] and 4 shares from [GMM⁺24].

As described in Section 2, we generate hierarchical netlists of our implementations for 2, 3 and 4 shares, annotate the glitch-robust PINI squaring gadgets and provide netlist and testbench to the MATCHI tool. As explained before we made sure that in all inputs (data and randomness) the 0-element is represented as p to guarantee the formally verified PINI security for version γ . MATCHI confirms that for all orders our implementations satisfy glitch+transition-robustness, as long as two independently shared input tuples are encrypted simultaneously and occupy the pipeline. If this is not the case (e.g., only a single shared tuple is encrypted), transitional leakages can reduce the security order. Our full implementations including the verification scripts for the mentioned tools are available at https://github.com/uclcrypto/prime_masking_verification.

4.2 Concurrent Randomness Generation

In [GMM⁺24] it is stated as an open problem to efficiently generate random numbers uniformly distributed over $\mathbb{F}_p$ for such masked cipher designs. Two different approaches are suggested, i) using rejection sampling in conjunction with a standard binary Random Number Generator (RNG) and ii) using an RNG that natively operates over $\mathbb{F}_p$. In this work we explore both options and compare their merits. While rejection sampling inevitably comes with constant-time- and/or buffering issues that need to be addressed by the hardware engineer, a native RNG over $\mathbb{F}_p$ first needs to be developed based on the specific choice of p . Hence, both approaches come with their own drawbacks.

4.2.1 Rejection Sampling + Standard PRNG

Unrolled implementations of the Trivium stream cipher [Can06] as well as its variants Bivium [Rad06] and Kreyvium [CCF⁺16] have been suggested for highly efficient concurrent randomness generation for hardware masking in [CMM⁺24]. All three ciphers produce keystream in the form of individual bits, i.e., $\mathbb{F}_2$ elements. When random values over $\mathbb{F}_{2^7-1}$ are needed, one may simply generate random chunks of 7 bits. Then, if all bits in a chunk are 1, the respective value is rejected as out of range. For our γ version of field arithmetic,

Table 4: Implementation figures for Bivium and Trivium with Rejection Sampling (RS) as well as Privium in 65 nm technology. Circuits can be clocked above 100 MHz at typical operating conditions (room temperature, nominal voltage). Power is estimated at 10 MHz.

PRNG	bits/cycle	Area [GE]				Power [mW]	Init. [cyc]
		PRNG Logic	PRNG FFs	RS Logic	RS FFs		
Bivium + RS	12 × 7	2050.00	840.75	2916.00	6783.00	0.2711	9+17
	30 × 7	7664.00	840.75	7290.00	16957.50	0.5858	4+17
	72 × 7	59180.25	840.75	17496.00	40698.00	1.6250	2+17
Trivium + RS	12 × 7	3073.00	1368.00	2916.00	6783.00	0.2910	14+17
	30 × 7	10341.50	1368.00	7290.00	16957.50	0.6190	6+17
	72 × 7	97440.75	1368.00	17496.00	40698.00	1.9289	3+17
Privium	12 × 7	7716.50	2394.00	0.00	0.00	0.1889	24
	30 × 7	18097.00	2394.25	0.00	0.00	0.3100	10
	72 × 7	47622.00	2398.25	0.00	0.00	0.6583	4

we reject chunks if all bits are 0 instead. Naturally, one may not predict (if the RNG is sound) when such an event occurs, thus some form of compensation is needed. Either a sufficiently large buffer of valid random elements needs to be kept available to draw from if needed. Or the implementation consuming the produced randomness has to be halted until sufficiently many good randoms are produced to continue securely. The probability that at least x out of y randomly chosen n -bit words are valid $\mathbb{F}_{2^n-1}$ elements is given by

$$Pr = \sum_{k=x}^y \binom{y}{k} \left(\frac{2^n - 1}{2^n} \right)^k \left(\frac{1}{2^n} \right)^{y-k}. \quad (2)$$

Each 7-bit bus for the randomness input of the masked cipher implementation needs to carry a freshly sampled field element in each of the 258 total clock cycles. 12, 30 and 72 such input buses of 7-bit elements are present in the **small-pSquare** implementations with 2, 3 and 4 shares, respectively. In order to keep the probability of having to stall the implementation or use a bad random in the masked implementation below a certain threshold, we need to ensure that each input bus is sufficiently buffered. When generating 272, 275 or 279 independent 7-bit values for each of the independent input buses over the duration of one masked double-encryption, we can keep the probability that any of the buses runs out of valid random field elements during a computation below 2^{-20} , 2^{-30} or 2^{-40} , respectively. For this, we simply attach a FIFO buffer of size 14, 17 or 21 field elements to each of the buses and only shift a fresh element in when it is valid, i.e., within range. For the 2^{-30} threshold, exemplarily, this approach leads to the cost and performance figures presented in Table 4. While the 2-share implementations perform decently, the randomness production for the higher-order masked implementation appears to become a bottleneck. The alternative to this buffering solution is of course a halt signal that stops the implementation whenever any of the randomness buses carries an out-of-range value. We do not evaluate such a non-constant-time solution in this work.

4.2.2 The Privium primitive

As the rejection sampling solution is relatively expensive, we now explore whether a custom RNG dedicated to the underlying finite field may not deliver more favorable cost and performance figures. For this purpose we have designed the **Privium** primitive which is depicted in Figure 5 and operates over a state of $\mathbb{F}_{2^7-1}^{72}$. Its design is inspired by Bivium and based on two coupled Non-Linear-Feedback Shift Registers (NLFSRs). Its words s_i are elements of the prime field $s_i \in \mathbb{F}_{2^7-1}$. The "+" symbol in Figure 5 denotes addition modulo p . Importantly, we can use the incomplete modulo reduction for the addition explained in Section 2 here, too, in order to obtain outputs that are uniformly distributed

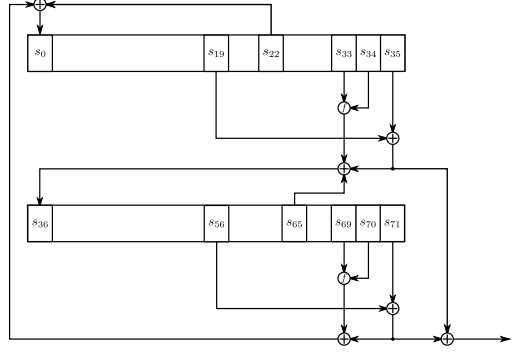


Figure 5: Schematic of **Privium**, a Bivium-inspired primitive consisting of 2 NLFSRs for concurrent randomness generation over $\mathbb{F}_{2^7-1}$.

over $\{1, \dots, p\}$ instead of $\{0, \dots, p-1\}$, which is secure in conjunction with version γ of the squaring gadgets. The function f is defined as $f(x, y) = \psi_7(x + y)$ with

$$\begin{aligned} \psi_7(x) = & 6 \cdot x^{125} + 60 \cdot x^{123} + 7 \cdot x^{121} + 42 \cdot x^{117} + 15 \cdot x^{115} + 85 \cdot x^{111} \\ & + 29 \cdot x^{109} + 49 \cdot x^{107} + 71 \cdot x^{103} + 56 \cdot x^{101} + 59 \cdot x^{97} + 62 \cdot x^{95} \\ & + 51 \cdot x^{93} + 12 \cdot x^{89} + 75 \cdot x^{87} + 29 \cdot x^{83} + 120 \cdot x^{81} + 100 \cdot x^{79} \\ & + 37 \cdot x^{75} + 45 \cdot x^{73} + 63 \cdot x^{69} + 61 \cdot x^{67} + 67 \cdot x^{65} + 43 \cdot x^{61} \\ & + 13 \cdot x^{59} + 108 \cdot x^{55} + 106 \cdot x^{53} + 87 \cdot x^{51} + 82 \cdot x^{47} + 8 \cdot x^{45} \\ & + 77 \cdot x^{41} + 95 \cdot x^{39} + 111 \cdot x^{37} + 29 \cdot x^{33} + 85 \cdot x^{31} + 49 \cdot x^{27} \\ & + 28 \cdot x^{25} + 72 \cdot x^{23} + 2 \cdot x^{19} + 51 \cdot x^{17} + 83 \cdot x^{13} + 70 \cdot x^{11} \\ & + 87 \cdot x^9 + 95 \cdot x^5 + 67 \cdot x^3 + 50 \cdot x \end{aligned}$$

This is the same function used in the tweak schedule of **small-pSquare**, defined in [GMM⁺24]. It provides strong algebraic properties and can be implemented very efficiently (namely, completely free in hardware) as a bit shuffle:

$$\psi_7\left(x = \sum_{i=0}^6 x_i \cdot 2^i\right) = x_0 \cdot 2^5 + x_1 \cdot 2^3 + x_2 \cdot 2^0 + x_3 \cdot 2^4 + x_4 \cdot 2^1 + x_5 \cdot 2^6 + x_6 \cdot 2^2.$$

Like Bivium/Trivium/Kreyvium, **Privium** can be unrolled to produce an arbitrary number of field elements as output stream per clock cycle. Unrolling can be performed in increments of up to 18 $\mathbb{F}_{2^7-1}$ elements ($18 \times 7 = 126$ bits) per cycle without increasing the critical path of the circuit (for Bivium/Trivium/Kreyvium the equivalent is 64 bits, see [CMM⁺24]). The initialization phase iterates $4 \times$ over the entire shift register, i.e., $4 \times 72 = 288$ rounds of updating a single $\mathbb{F}_{2^7-1}$ element. The concrete unrolling factors for our masked serialized pipelined implementations of **small-pSquare** (Figure 4) with 2, 3 and 4 shares are 12, 30 and 72 $\mathbb{F}_{2^7-1}$ elements, respectively. For these unrolling factors, the initialization of **Privium** takes 24, 10 and 4 cycles, respectively, to complete. We have verified that the output stream of **Privium** successfully passes all tests of NIST’s 800-22 test suite for random and Pseudo-Random Number Generators (PRNGs) for cryptographic applications. Table 4 shows that the use of **Privium** is always preferable over the binary primitives with rejection sampling for our selected probability threshold and the gap increases at higher orders. This might change if solutions to stall the masked implementation are implemented or if out-of-range random values are tolerated more often at some security risk.

We stress that we do not provide any in-depth black-box security analysis of this primitive in this work, and explicitly do not suggest the use of **Privium** beyond the generation of masking randomness, especially not as a cryptographically strong PRNG. As discussed in [CMM⁺24], in the setting of masking randomness generation, adversaries

Table 5: Pre- and post-layout implementation figures for F-serialized pipelined masked small-pSquare using Privium as PRNG for concurrent randomness generation. Synthesized and implemented for 50 MHz operation under worst-case conditions (high temperature, low voltage). Evaluated at typical operating conditions (room temperature, nominal voltage). Power is estimated at 10 MHz.

	d	Area [GE]			Power [mW]	Delay [ns]	Cycles #
		small-pSquare	Privium	Total			
Pre-layout	2	27377.00	10115.75	42907.25	0.4518	10.10	(24+258) / 2
	3	48685.50	20486.50	76843.00	0.8253	10.29	(10+258) / 2
	4	72282.50	49856.50	132095.25	1.4360	10.25	(4+258) / 2
Post-layout	2	29048.50	10626.50	45760.50	0.4252	9.02	(24+258) / 2
	3	51082.50	20696.00	80480.75	0.8185	8.80	(10+258) / 2
	4	75617.25	50336.25	137443.75	1.4799	9.05	(4+258) / 2

cannot observe the generated output stream but at most the leakage of it. This makes cryptanalytic state recovery based on generated outputs significantly more difficult and puts lighter requirements on the cryptographic strength. For this reason, Bivium is also currently assumed to be sufficiently secure for this purpose (e.g., used in OpenTitan⁸). We conjecture that the cryptanalytic security and leakage-resilience of Privium is stronger than that of Bivium. This is due both to its larger state size (504 vs. 177 bits) and due to the fact that for frequently observed (e.g., Hamming weight or similar) leakage functions, state recovery attacks against prime-based LFSRs are expected to be considerably harder than against their binary counterparts. This is supported by arguments similar to those on hard physical learning problems like Learning With Physical Rounding [DMMS21, HMM⁺23]. We leave the further analysis of this conjecture as an interesting scope for further research.

4.3 ASIC Implementation Figures

Now that we have provably glitch+transition-robust cipher implementations as well as a random number generator producing the required output distribution at our disposal, it is time to merge them into fully integrated circuits and realize them on actual silicon. Functionally, the merge requires only an additional state machine that makes sure that the entire Privium initialization is performed before the masked cipher starts to execute. Beyond that, the Privium outputs can be directly connected to the randomness inputs of the masked ciphers. Table 5 reports the resulting cost and performance numbers after synthesis (i.e., pre-layout) and after place&route (i.e., post-layout) of our ASIC prototype. Area figures are reported separately for small-pSquare, Privium and their combination. The chip is designed for ≥ 50 MHz operation under worst-case conditions (high temperature, low voltage). The delay numbers are reported for typical conditions, indicating that frequencies up to 100 MHz are feasible for the post-layout circuits. The reported power consumption is estimated at 10 MHz for consistency.

The full ASIC prototype has been designed and manufactured in 65 nm Complementary Metal-Oxide-Semiconductor (CMOS) technology. Its layout is depicted in Figure 6a. The die is 2×2 millimeters in size and contains 9 metal layers of which the topmost is only used for logo/orientation-purposes and not for functional routing. On each of its 4 sides it has 7 logical IO cells and 4 additional power delivery cells to supply the standard-cell core and the IO ring with V_{DD} and GND respectively, resulting in 11 bond pads on each side and 44 bond pads in total. With these parameters it fits nicely into the common JLCC44 package, as shown in Figure 6b. The custom circuit board shown in Figure 6c has been optimized for low-noise power measurements and electrically decouples the control Field Programmable Gate Array (FPGA) from the target ASIC. Logically, the two parts of the

⁸https://opentitan.org/book/hw/ip/kmac/doc/theory_of_operation.html

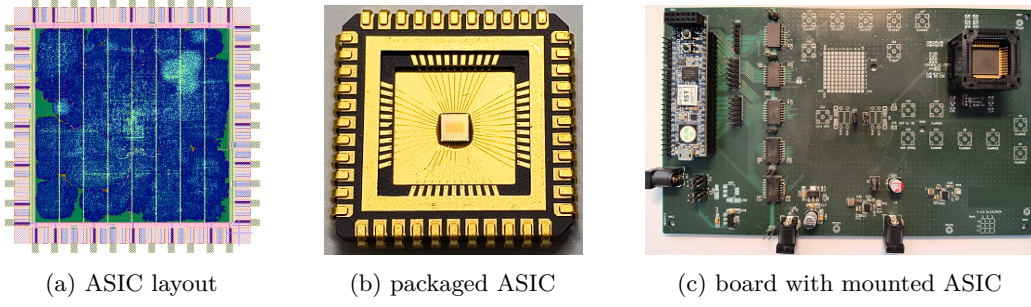
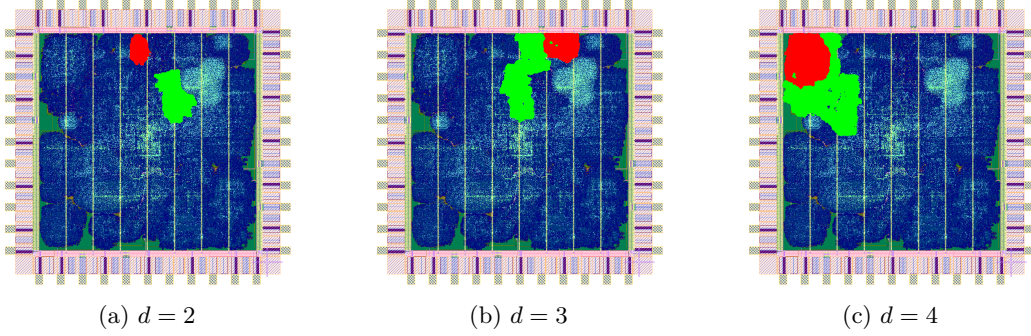


Figure 6: Layout and photos of chip and board.

Figure 7: Floorplan locations of the different masked **small-pSquare** cores with d shares respectively, marked in **green**, with the associated **Privium** locations marked in **red**.

board are connected through ADuM260N 6-channel digital isolators.

The approximate floorplan locations of the three different masked **small-pSquare** cores with $d = 2, 3$ and 4 shares are highlighted in Figures 7a, 7b and 7c, respectively, in **green** color. The corresponding **Privium** instances with unrolling factors 12 (left), 30 (middle) and 72 (right) $\mathbb{F}_p$ elements are highlighted in **red** color. As also apparent in Table 5, the **Privium** cores for concurrent randomness generation occupy an increasingly large share of the total area. This could potentially be offset by using multiple instances with smaller unrolling factors instead of one large instance, however, that can make the seeding more complicated [CMM⁺24].

5 Physical Evaluation

After fabrication of the ASIC and the PCB, we are now in position to perform an experimental analysis of the security properties of the higher-order masked **small-pSquare** implementations. We limit ourselves to a leakage detection experiment in this work and leave an attack-based evaluation for future work.

Setup. The control FPGA on the custom PCB shown in Figure 6c is used to facilitate communication between computer and target ASIC. It is a Xilinx Artix-7 module and part of the Digilent Cmod A7-15T breadboardable 48-pin DIP form factor board. The board is also used to supply the target ASIC with a stable 12 MHz clock. All cores and controllers are clock-gated, including their respective parts of the clock tree and can be activated/deactivated individually on demand. We measure the power consumption of the ASIC using a PicoScope 5244D digital sampling oscilloscope at 250 MS/s sampling rate (resulting in $\frac{250}{12} \approx 20.833$ samples per clock cycle) with 12-bit vertical resolution through

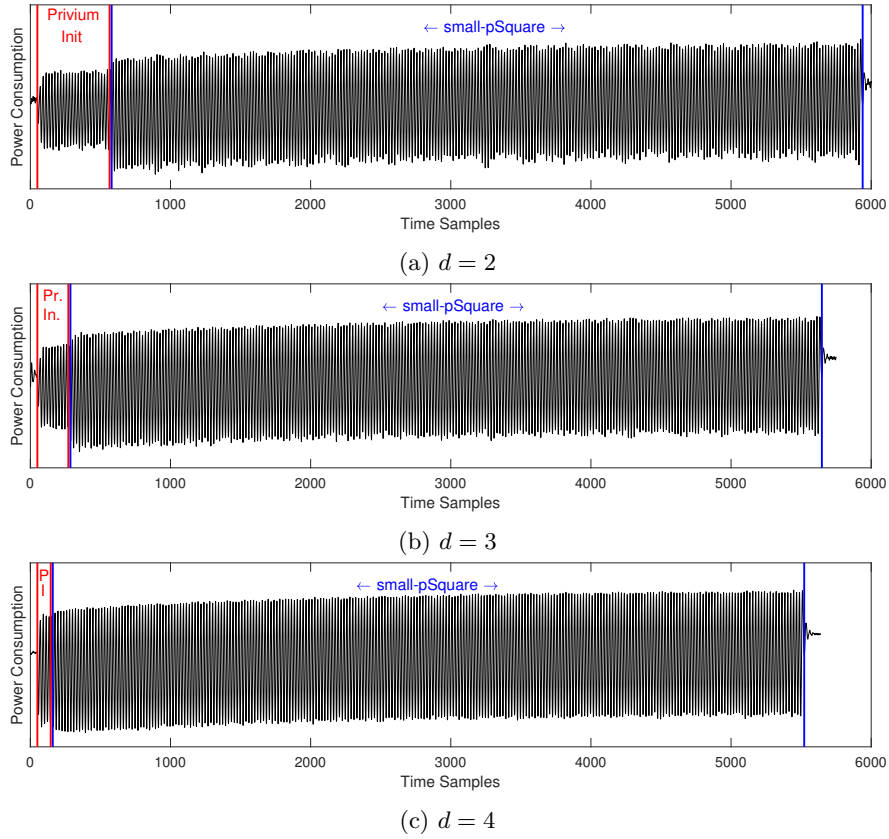


Figure 8: Sample traces measured from the masked **small-pSquare** ASIC cores with d shares. Each clock cycle corresponds to ≈ 21 time samples. The **Privium** initialization takes fewer clock cycles for the larger instances, as they achieve 4 state rotations faster.

a Tektronix CT-1 current probe (bandwidth of up to 1 GHz) placed in the power supply path of the ASIC.

Core Execution and Measurements. Figure 8 shows sample measurements of the masked **small-pSquare** ASIC cores with $d = 2, 3, 4$ shares and integrated **Privium** for randomness generation. The initialization phase of **Privium** is marked in **red** and can clearly be distinguished by the signal amplitude from the start of the concurrent execution of **small-pSquare** and **Privium**, marked in **blue**. Since the higher-order masked cipher implementations use RNG instances with larger unrolling factor, they complete the initialization phase faster, as 4 state rotations are completed sooner, and therefore lead to shorter trace lengths. The F-serialized two-stage pipelined implementations of **small-pSquare** require the same amount of cycles for all numbers of shares considered here (see also Table 5).

Test Vector Leakage Assessment (TVLA). We perform TVLA for leakage detection with the common threshold of $\text{abs}(t) > 4.5$ [SM15]. The results are presented for 100 million traces each in Figure 9. While leakage detection using such univariate statistical hypothesis tests enables only limited conclusions about the actual vulnerability of implementations to concrete attacks, it is well-suited to confirm the minimum statistical order of present leakages and potentially identify (at least significant) masking flaws. In our results, the only conclusively detectable leakage is present for the 2-share implementation in the second order, as expected. Neither the 3-share, nor the 4-share implementation exhibit

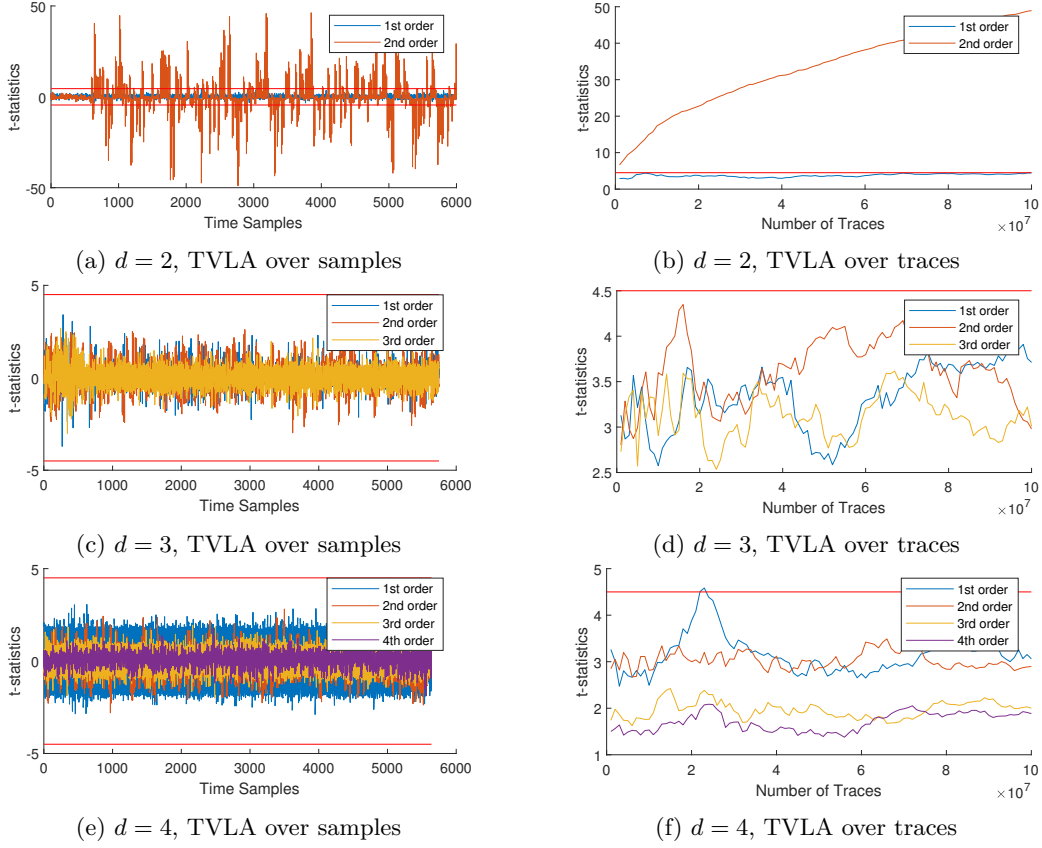


Figure 9: Test Vector Leakage Assessment (TVLA) results of the three different masked small-pSquare ASIC implementations.

any conclusively detectable leakage at the evaluated statistical moments. We believe this result indicates a desirable growth of side-channel resistance in the protection order.

6 Conclusion

In this work we have presented our findings from formally verifying, securely integrating and physically evaluating higher-order masked implementations of **small-pSquare** on real silicon. Some parts of our results are generalizable to any prime-field masked circuit (e.g., the masked gadget/circuit verification), while others are specific to the Mersenne-prime based squaring gadgets that have been used for protected cipher implementations in this and contemporary works (e.g., the 0-issue and the associated fixes). We have devised a technique to certify the composability properties of gadgets defined over $\mathbb{F}_p$ using existing $\mathbb{F}_2$ -focused formal verification tools and proved its soundness. Using this technique, together with existing composition-level and simulation-based verification tools, we have identified security issues resulting from incomplete modulo reductions in public source codes and discussed how to repair them in the most cost effective way. For concurrent randomness generation of masking schemes defined over $\mathbb{F}_{2^7-1}$ we have introduced the **Privium** primitive which is optimized for unrolled implementation. Our ASIC-based implementation details and physical security analysis provide new insights and show the promising tradeoff between physical security and performance that higher-order masked implementations of **small-pSquare** can provide. For about 50k gate equivalents (80k including randomness generator and integration), our 3-share implementation performs two masked encryptions

within 258 clock cycles without detectable leakage at any order, up to *at least* 100 million traces. For future work, investigating the concrete security of the ASIC cores against profiled horizontal attacks and statistical ineffective fault attacks could be valuable, as both are types of adversaries that prime-field masking claims improved resistance against.

Acknowledgments

Gaëtan Cassiers and Thorben Moos are postdoctoral researchers of the Belgian Fund for Scientific Research (F.R.S.-FNRS), François-Xavier Standaert is a research director of the F.R.S.-FNRS. This work has been funded in part by the European Union through the ERC projects 101077506 (acronym AMaskZONE) and 101096871 (acronym BRIDGE), and by the German Research Foundation (DFG) through the project 549340884 (MatSec). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

- [BBB⁺21] Anubhab Baksi, Shivam Bhasin, Jakub Breier, Mustafa Khairallah, Thomas Peyrin, Sumanta Sarkar, and Siang Meng Sim. DEFAULT: cipher level resistance against differential fault attack. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part II*, volume 13091 of *Lecture Notes in Computer Science*, pages 124–156. Springer, 2021.
- [BBC⁺19] Gilles Barthe, Sonia Belaïd, Gaëtan Cassiers, Pierre-Alain Fouque, Benjamin Grégoire, and François-Xavier Standaert. maskverif: Automated verification of higher-order masking in presence of physical defaults. In Kazue Sako, Steve A. Schneider, and Peter Y. A. Ryan, editors, *Computer Security - ESORICS 2019 - 24th European Symposium on Research in Computer Security, Luxembourg, September 23-27, 2019, Proceedings, Part I*, volume 11735 of *Lecture Notes in Computer Science*, pages 300–318. Springer, 2019.
- [BBC⁺20] Davide Bellizia, Olivier Bronchain, Gaëtan Cassiers, Vincent Grosso, Chun Guo, Charles Momin, Olivier Pereira, Thomas Peters, and François-Xavier Standaert. Mode-level vs. implementation-level physical security in symmetric cryptography - A practical guide through the leakage-resistance jungle. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 369–400. Springer, 2020.
- [BBD⁺16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong non-interference and type-directed higher-order masking. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 116–129. ACM, 2016.

- [BBK⁺13] Begül Bilgin, Andrey Bogdanov, Miroslav Knezevic, Florian Mendel, and Qingju Wang. Fides: Lightweight authenticated cipher with side-channel resistance for constrained hardware. In Guido Bertoni and Jean-Sébastien Coron, editors, *Cryptographic Hardware and Embedded Systems - CHES 2013 - 15th International Workshop, Santa Barbara, CA, USA, August 20-23, 2013. Proceedings*, volume 8086 of *Lecture Notes in Computer Science*, pages 142–158. Springer, 2013.
- [BBP⁺16] Sonia Belaïd, Fabrice Benhamouda, Alain Passelègue, Emmanuel Prouff, Adrian Thillard, and Damien Vergnaud. Randomness complexity of private circuits for multiplication. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 616–648. Springer, 2016.
- [BFG⁺25] Christof Beierle, Jakob Feldtkeller, Anna Guinet, Tim Güneysu, Gregor Leander, Jan Richter-Brockmann, and Pascal Sasdrich. INDIANA - verifying (random) probing security through indistinguishability analysis. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques Madrid, Spain, May 4-8, 2025, Proceedings, Part VIII*, volume 15608 of *Lecture Notes in Computer Science*, pages 33–63. Springer, 2025.
- [BGI⁺18] Roderick Bloem, Hannes Groß, Rinat Iusupov, Bettina Könighofer, Stefan Mangard, and Johannes Winter. Formal verification of masked hardware implementations in the presence of glitches. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 321–353. Springer, 2018.
- [BGM⁺25] Briec Balon, Lorenzo Grassi, Pierrick Méaux, Thorben Moos, François-Xavier Standaert, and Matthias Johann Steiner. mid-psquare: Leveraging the strong side-channel security of prime-field masking in software. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(4):486–519, 2025.
- [BGR18] Sonia Belaïd, Dahmun Goudarzi, and Matthieu Rivain. Tight private circuits: Achieving probing security with the least refreshing. In Thomas Peyrin and Steven D. Galbraith, editors, *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 343–372. Springer, 2018.
- [BLMR19] Christof Beierle, Gregor Leander, Amir Moradi, and Shahram Rasoolzadeh. CRAFT: lightweight tweakable block cipher with efficient protection against DFA attacks. *IACR Trans. Symmetric Cryptol.*, 2019(1):5–45, 2019.
- [BMRT22] Sonia Belaïd, Darius Mercadier, Matthieu Rivain, and Abdul Rahman Taleb. Ironmask: Versatile verification of masking security. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*, pages 142–160. IEEE, 2022.

- [Can06] Christophe De Cannière. Trivium: A stream cipher construction inspired by block cipher design principles. In Sokratis K. Katsikas, Javier López, Michael Backes, Stefanos Gritzalis, and Bart Preneel, editors, *Information Security, 9th International Conference, ISC 2006, Samos Island, Greece, August 30 - September 2, 2006, Proceedings*, volume 4176 of *Lecture Notes in Computer Science*, pages 171–186. Springer, 2006.
- [Cas25] Gaëtan Cassiers. MATCHI: formal verification of hardware private circuits. *IACR Cryptol. ePrint Arch.*, 2025:1882, 2025.
- [CCF⁺16] Anne Canteaut, Sergiu Carpov, Caroline Fontaine, Tancrede Lepoint, María Naya-Plasencia, Pascal Paillier, and Renaud Sirdey. Stream ciphers: A practical solution for efficient homomorphic-ciphertext compression. In Thomas Peyrin, editor, *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers*, volume 9783 of *Lecture Notes in Computer Science*, pages 313–333. Springer, 2016.
- [CGLS21] Gaëtan Cassiers, Benjamin Grégoire, Itamar Levi, and François-Xavier Standaert. Hardware private circuits: From trivial composition to full verification. *IEEE Trans. Computers*, 70(10):1677–1690, 2021.
- [CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 398–412. Springer, 1999.
- [CMM⁺23] Gaëtan Cassiers, Loïc Masure, Charles Momin, Thorben Moos, and François-Xavier Standaert. Prime-field masking in hardware and its soundness against low-noise SCA attacks. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(2):482–518, 2023.
- [CMM⁺24] Gaëtan Cassiers, Loïc Masure, Charles Momin, Thorben Moos, Amir Moradi, and François-Xavier Standaert. Randomness generation for secure hardware masking - unrolled trivium to the rescue. *IACR Commun. Cryptol.*, 1(2):4, 2024.
- [CS20] Gaëtan Cassiers and François-Xavier Standaert. Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE Trans. Inf. Forensics Secur.*, 15:2542–2555, 2020.
- [CS21] Gaëtan Cassiers and François-Xavier Standaert. Provably secure hardware masking in the transition- and glitch-robust probing model: Better safe than sorry. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(2):136–158, 2021.
- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *J. Cryptol.*, 34(3):33, 2021.
- [DFS16] Stefan Dziembowski, Sebastian Faust, and Maciej Skórski. Optimal amplification of noisy leakages. In Eyal Kushilevitz and Tal Malkin, editors, *Theory of Cryptography - 13th International Conference, TCC 2016-A, Tel Aviv, Israel, January 10-13, 2016, Proceedings, Part II*, volume 9563 of *Lecture Notes in Computer Science*, pages 291–318. Springer, 2016.

- [DMMS21] Sébastien Duval, Pierrick Méaux, Charles Momin, and François-Xavier Standaert. Exploring crypto-physical dark matter and learning with physical rounding towards secure and efficient fresh re-keying. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(1):373–401, 2021.
- [dPKPR24] Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part I*, volume 14920 of *Lecture Notes in Computer Science*, pages 409–444. Springer, 2024.
- [dPPRS23] Rafaël del Pino, Thomas Prest, Mélissa Rossi, and Markku-Juhani O. Saarinen. High-order masking of lattice signatures in quasilinear time. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 1168–1185. IEEE, 2023.
- [FGP⁺18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable masking schemes in the presence of physical defaults & the robust probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):89–120, 2018.
- [FMM⁺24] Sebastian Faust, Loïc Masure, Elena Micheli, Maximilian Ortl, and François-Xavier Standaert. Connecting leakage-resilient secret sharing to practice: Scaling trends and physical dependencies of prime field masking. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part IV*, volume 14654 of *Lecture Notes in Computer Science*, pages 316–344. Springer, 2024.
- [FMM⁺25] Sebastian Faust, Loïc Masure, Elena Michel, Hai Hoang Nguyen, Maximilian Ortl, and François-Xavier Standaert. IP masking with generic security guarantees under minimum assumptions, and applications. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part II*, *Lecture Notes in Computer Science*, pages 544–575. Springer, 2025.
- [GGNS13] Benoît Gérard, Vincent Grosso, María Naya-Plasencia, and François-Xavier Standaert. Block ciphers that are easier to mask: How far can we go? In Guido Bertoni and Jean-Sébastien Coron, editors, *Cryptographic Hardware and Embedded Systems - CHES 2013 - 15th International Workshop, Santa Barbara, CA, USA, August 20-23, 2013. Proceedings*, volume 8086 of *Lecture Notes in Computer Science*, pages 383–399. Springer, 2013.
- [GHP⁺21] Barbara Gigerl, Vedad Hadzic, Robert Primas, Stefan Mangard, and Roderick Bloem. Coco: Co-design and co-verification of masked software implementations on cpus. In Michael D. Bailey and Rachel Greenstadt, editors, *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 1469–1468. USENIX Association, 2021.
- [GLSV14] Vincent Grosso, Gaëtan Leurent, François-Xavier Standaert, and Kerem Varici. Ls-designs: Bitslice encryption for efficient masked software implementations.

- In Carlos Cid and Christian Rechberger, editors, *Fast Software Encryption - 21st International Workshop, FSE 2014, London, UK, March 3-5, 2014. Revised Selected Papers*, volume 8540 of *Lecture Notes in Computer Science*, pages 18–37. Springer, 2014.
- [GMM16] Daniel Gruss, Clémentine Maurice, and Stefan Mangard. Rowhammer.js: A remote software-induced fault attack in javascript. In Juan Caballero, Urko Zurutuza, and Ricardo J. Rodríguez, editors, *Detection of Intrusions and Malware, and Vulnerability Assessment - 13th International Conference, DIMVA 2016, San Sebastián, Spain, July 7-8, 2016, Proceedings*, volume 9721 of *Lecture Notes in Computer Science*, pages 300–321. Springer, 2016.
- [GMM⁺24] Lorenzo Grassi, Loïc Masure, Pierrick Méaux, Thorben Moos, and François-Xavier Standaert. Generalized feistel ciphers for efficient prime field masking. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part III*, volume 14653 of *Lecture Notes in Computer Science*, pages 188–220. Springer, 2024.
- [HB21] Vedad Hadzic and Roderick Bloem. COCOALMA: A versatile masking verifier. In *Formal Methods in Computer Aided Design, FMCAD 2021, New Haven, CT, USA, October 19-22, 2021*, pages 1–10. IEEE, 2021.
- [HMM⁺23] Clément Hoffmann, Pierrick Méaux, Charles Momin, Yann Rotella, François-Xavier Standaert, and Balazs Udvarhelyi. Learning with physical rounding for linear and quadratic leakage functions. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 410–439. Springer, 2023.
- [ISW03] Yuval Ishai, Amit Sahai, and David A. Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003.
- [JKP24] Amit Jana, Anup Kumar Kundu, and Goutam Paul. More vulnerabilities of linear structure sbox-based ciphers reveal their inability to resist DFA. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VIII*, volume 15491 of *Lecture Notes in Computer Science*, pages 168–203. Springer, 2024.
- [KMMS22] David Knichel, Amir Moradi, Nicolai Müller, and Pascal Sasdrich. Automated generation of masked hardware. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):589–629, 2022.
- [KSM20] David Knichel, Pascal Sasdrich, and Amir Moradi. SILVER - statistical independence and leakage verification. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*,

- volume 12491 of *Lecture Notes in Computer Science*, pages 787–816. Springer, 2020.
- [KSM22] David Knichel, Pascal Sasdrich, and Amir Moradi. Generic hardware private circuits towards automated generation of composable secure gadgets. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):323–344, 2022.
- [MCS22] Charles Momin, Gaëtan Cassiers, and François-Xavier Standaert. Handcrafting: Improving automated masking in hardware with manual optimizations. In Josep Balasch and Colin O’Flynn, editors, *Constructive Side-Channel Analysis and Secure Design - 13th International Workshop, COSADE 2022, Leuven, Belgium, April 11-12, 2022, Proceedings*, volume 13211 of *Lecture Notes in Computer Science*, pages 257–275. Springer, 2022.
- [MDB21] Macarena C. Martínez-Rodríguez, Ignacio M. Delgado-Lozano, and Billy Bob Brumley. Sok: Remote power analysis. In Delphine Reinhardt and Tilo Müller, editors, *ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, August 17-20, 2021*, pages 7:1–7:12. ACM, 2021.
- [MKSM22] Nicolai Müller, David Knichel, Pascal Sasdrich, and Amir Moradi. Transitional leakage in theory and practice unveiling security flaws in masked circuits. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(2):266–288, 2022.
- [MM22] Nicolai Müller and Amir Moradi. PROLEAD A probing-based hardware leakage detection tool. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):311–348, 2022.
- [MMMS23] Loïc Masure, Pierrick Méaux, Thorben Moos, and François-Xavier Standaert. Effective and efficient masking with low noise using small-mersenne-prime ciphers. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part IV*, volume 14007 of *Lecture Notes in Computer Science*, pages 596–627. Springer, 2023.
- [MSS24] Thorben Moos, Sayandeep Saha, and François-Xavier Standaert. Prime masking vs. faults - exponential security amplification against selected classes of attacks. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(4):690–736, 2024.
- [NDE22] Marcel Nageler, Christoph Dobraunig, and Maria Eichlseder. Information-combining differential fault attacks on DEFAULT. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 168–191. Springer, 2022.
- [PRC12] Gilles Piret, Thomas Roche, and Claude Carlet. PICARO - A block cipher allowing efficient higher-order side-channel resistance. In Feng Bao, Pierangela Samarati, and Jianying Zhou, editors, *Applied Cryptography and Network Security - 10th International Conference, ACNS 2012, Singapore, June 26-29, 2012. Proceedings*, volume 7341 of *Lecture Notes in Computer Science*, pages 311–328. Springer, 2012.

- [Rad06] Havard Raddum. Cryptanalytic results on triviality. *eSTREAM, ECRYPT Stream Cipher Project, Report 2006/039*, 2006.
- [RFSG22] Jan Richter-Brockmann, Jakob Feldtkeller, Pascal Sasdrich, and Tim Güneysu. VERICA - verification of combined attacks automated formal verification of security against simultaneous information leakage and tampering. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):255–284, 2022.
- [sak] Side-channel AttacK User Reference Architecture. <http://satoh.cs.uec.ac.jp/SAKURA/index.html>.
- [SBD⁺20] Thierry Simon, Lejla Batina, Joan Daemen, Vincent Grosso, Pedro Maat Costa Massolino, Kostas Papagiannopoulos, Francesco Regazzoni, and Niels Samwel. Friet: An authenticated encryption scheme with built-in fault detection. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology - EURO-CRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 581–611. Springer, 2020.
- [SM15] Tobias Schneider and Amir Moradi. Leakage assessment methodology - A clear roadmap for side-channel evaluations. In Tim Güneysu and Helena Handschuh, editors, *Cryptographic Hardware and Embedded Systems - CHES 2015 - 17th International Workshop, Saint-Malo, France, September 13-16, 2015, Proceedings*, volume 9293 of *Lecture Notes in Computer Science*, pages 495–513. Springer, 2015.
- [UHIM24] Rei Ueno, Naofumi Homma, Akiko Inoue, and Kazuhiko Minematsu. Sok: Pseudorandom generation for masked cryptographic implementation. *IACR Cryptol. ePrint Arch.*, page 1968, 2024.
- [WFH⁺22] Senpeng Wang, Dengguo Feng, Bin Hu, Jie Guan, and Tairong Shi. Practical attacks on full-round FRIET. *IACR Trans. Symmetric Cryptol.*, 2022(4):105–119, 2022.
- [ZCF25] Feng Zhou, Hua Chen, and Limin Fan. Prover - toward more efficient formal verification of masking in probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(1):552–585, 2025.
- [ZMM23] Jannik Zeitschner, Nicolai Müller, and Amir Moradi. Prolead_sw probing-based software leakage detection for ARM binaries. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(3):391–421, 2023.